

Funções I: parâmetros, chamadas e return

Atividade 1 · Definir uma vez, chamar várias vezes

Questões de autoavaliação

1. Registrar as saídas, identificar parâmetros e argumentos e contar as execuções do corpo da função.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

O exemplo apresenta Início, Olá Ana, Olá Rui e Fim. Ao acrescentar uma chamada antes do print final, surge mais uma saudação nesse ponto. O corpo passa de duas para três execuções.

```
def saudar(nome, curso):  
    print('Olá', nome, '- curso:', curso)  
  
print('Início')  
saudar('Ana', 'Informática')  
saudar('Rui', 'Informática')  
saudar('Eva', 'Informática')  
print('Fim')
```

nome e curso são parâmetros. Na primeira chamada, Ana e Informática são argumentos. Os nomes pessoais usados pelos alunos podem variar.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 2 · Valores predefinidos e argumentos nomeados

Questões de autoavaliação

1. Identificar a chamada que mudou de resultado. Explicar a diferença entre omitir a taxa e fornecer um argumento explícito.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Resultados iniciais: 90.0, 80.0 e 95.0. Com a taxa predefinida 0.15: 85.0, 80.0 e 95.0. Só a chamada que omite taxa usa o novo valor predefinido. Com taxa=0, o resultado é 100.

```
def aplicar_desconto(preco, taxa=0.15):  
    return preco * (1 - taxa)  
  
print(aplicar_desconto(100))  
print(aplicar_desconto(100, 0.20))  
print(aplicar_desconto(taxa=0.05, preco=100))  
print(aplicar_desconto(100, taxa=0))
```

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Funções I: parâmetros, chamadas e return

Atividade 3 · Mostrar ou devolver

Questões de autoavaliação

1. Comparar x e y antes e depois. Identificar a impressão que desaparece e explicar a reutilização do valor devolvido.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: a função `mostrar_dobro` imprime 8; depois surgem x: None e y: 8. O `print(y + 3)` acrescenta 11.

Ao substituir `print(n * 2)` por `return n * 2` em `mostrar_dobro`, x passa a valer 8. Desaparece a impressão isolada de 8 dentro dessa função; continuam x: 8, y: 8 e 11. A atribuição recebe o valor devolvido pela chamada.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 4 · Uma função com decisão

Questões de autoavaliação

1. Registrar os resultados, explicar a igualdade e esclarecer se o segundo return executa depois do primeiro.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

```
def maior(a, b):  
    if a >= b:  
        return a  
    return b
```

```
def menor(a, b):  
    if a <= b:  
        return a  
    return b
```

```
print(maior(8, 3), menor(8, 3))  
print(maior(2, 9), menor(2, 9))  
print(maior(5, 5), menor(5, 5))
```

Saídas: 8 3; 9 2; 5 5. Em caso de igualdade, o valor devolvido é 5 em ambas. O primeiro return termina a chamada, pelo que o segundo não executa nessa chamada.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Funções I: parâmetros, chamadas e return

Atividade 5 · Aplicação integrada

Questões de autoavaliação

1. Explicar a organização do algoritmo e as decisões tomadas.
2. Comparar resultados previstos e observados em todos os testes pedidos.
3. Identificar uma falha possível e indicar como um teste a poderia revelar.

Resolução e verificação

```
def area(base, altura):  
    return base * altura  
  
def perimetro(base, altura):  
    return 2 * (base + altura)  
  
base = float(input('Base: '))  
altura = float(input('Altura: '))  
print('Área:', area(base, altura))  
print('Perímetro:', perimetro(base, altura))
```

5 e 3: área 15.0, perímetro 16.0. 4 e 4: área 16.0, perímetro 16.0. 0 e 3: área 0.0, perímetro 6.0. O último é um caso-limite numérico aceite pelo enunciado. Cada função calcula uma grandeza; a interação fica no código principal.

Funções I: parâmetros, chamadas e return

Atividade 6 · Um return demasiado cedo

Questões de autoavaliação

1. Por que o ciclo inicial termina cedo?
2. Por que a chamada com zero devolve None na versão inicial?
3. Quais os resultados corretos e o valor de `2 * soma_ate(4)`?

Resolução e verificação

```
def soma_ate(n):  
    total = 0  
    for valor in range(1, n + 1):  
        total += valor  
    return total  
  
print(soma_ate(0))  
print(soma_ate(1))  
print(soma_ate(4))  
print(2 * soma_ate(4))
```

A versão inicial devolve 1 para `n=4` e None para `n=0`. O return dentro do ciclo termina a primeira iteração; com zero, não há iterações nem return executado. Correto: 0,1,10 e 20. Devolver fora do ciclo cobre também a sequência vazia.

Funções I: parâmetros, chamadas e return

Atividade 7 · Descontos sucessivos não são somados

Questões de autoavaliação

1. Quais são os resultados das versões somada e sucessiva no primeiro teste?
2. Que valor recebe a segunda chamada a desconto?
3. Quais os resultados dos restantes testes?

Resolução e verificação

```
def desconto(preco, taxa):  
    return preco * (1 - taxa)  
  
def preco_final(preco, taxa1, taxa2):  
    intermedio = desconto(preco, taxa1)  
    return desconto(intermedio, taxa2)  
  
print(preco_final(100, 0.10, 0.20))  
print(preco_final(100, 0, 0.20))  
print(preco_final(80, 0.25, 0.25))
```

Somar as taxas dá 70.0; aplicar sucessivamente dá 72.0 porque a segunda chamada recebe 90.0. Restantes resultados: 80.0 e 45.0. Cada taxa incide sobre a base recebida pela função, não necessariamente sobre o preço inicial.

Funções I: parâmetros, chamadas e return

Atividade 8 · Decompor um orçamento

Questões de autoavaliação

1. Que responsabilidade tem cada função?
2. Quais os valores intermédios e finais do primeiro teste?
3. Como testar aplicar_iva isoladamente e como verificar a quantidade zero?

Resolução e verificação

```
def subtotal(preco, quantidade):  
    return preco * quantidade  
  
def aplicar_desconto(valor, taxa):  
    return valor * (1 - taxa)  
  
def aplicar_iva(valor, taxa):  
    return valor * (1 + taxa)  
  
def orcamento(preco, quantidade, desconto, iva):  
    bruto = subtotal(preco, quantidade)  
    liquido = aplicar_desconto(bruto, desconto)  
    return aplicar_iva(liquido, iva)  
  
print(orcamento(20, 3, 0.10, 0.23))  
print(orcamento(20, 0, 0.10, 0.23))  
print(orcamento(20, 3, 0, 0))
```

Primeiro teste: subtotal 60; após desconto 54; com IVA 66.42, admitindo a representação aproximada habitual dos decimais. Outros totais: 0.0 e 60. Teste isolado possível: aplicar_iva(100,0.23) deve produzir 123. A função orcamento coordena; as restantes efetuam um cálculo cada.

Critério de autoavaliação: 2 pontos quando a implementação cumpre as regras, passa os testes pedidos e as respostas explicam os resultados; 1 ponto se falta parte da solução, dos testes ou da justificação; 0 pontos se a tarefa permanece por resolver. Aplicar o mesmo critério às restantes atividades, num total de 16 pontos. A pontuação orienta o estudo e não constitui classificação formal.